

Rapport Projet Base de Données

MEMBRES DU BINÔME :

- Mulier Armand
- Metahri Ramy

Année : L2 Informatique

Date : Décembre 2025

TABLE DES MATIÈRES :

1. Présentation du Projet et Manuel Utilisateur
2. Section Technique : Conception de la Base de Données (Étapes 1 & 2)
3. Description Technique et Fonctionnalités SQL Avancées
4. Pistes d'Améliorations
5. Organisation du Travail

1.1 PRÉSENTATION GÉNÉRALE DU PROJET :

Dans le cadre de notre premier projet de base de données en deuxième année de licence informatique, nous avons été amenés à travailler sur le sujet SPA.

Le document de projet écrit était structuré en deux parties principales. La première partie visait à identifier et à analyser les besoins et les attentes du client afin de concevoir une base de données répondant à ses exigences spécifiques. Le sujet initial nous a fourni une base conceptuelle pour la base de données finale. Au cours du projet, nous avons reçu des directives complémentaires du client, nous demandant d'intégrer des fonctionnalités additionnelles à la base de données.

La deuxième partie du projet portait sur les exigences du client concernant le développement d'un site web, qui servirait d'interface utilisateur à la base de données. Cette interface permettrait aux utilisateurs d'interroger la base de données et, selon leurs droits d'accès, d'effectuer des modifications.

1.2. MANUEL UTILISATEUR SIMPLIFIÉ

Partie Publique :

- Accueil & Recherche : Dès l'arrivée sur le site, une carte interactive (OpenStreetMap) affiche tous les refuges partenaires. Une barre de recherche permet de filtrer par ville ou code postal. Possibilité de renseigner seulement le début du code postal (ex : 75). Il est également possible de cliquer sur un pin se situant sur la mini carte pour voir apparaître en surbrillance le refuge concerné...
- Consultation : En cliquant sur un refuge, l'utilisateur accède à la liste des animaux présents, avec pagination si la liste est longue. Des filtres permettent de trier par nom, espèce ou âge...

Partie Administration (Espace Staff) :

L'accès se fait via un login sécurisé.

- Tableau de Bord (Dashboard) : Une vue d'ensemble affiche les statistiques clés (taux d'occupation, alertes vétérinaires urgentes).
- Gestion des Pensionnaires :
 - Ajout : Un formulaire complet permet d'enregistrer un nouvel animal (génération automatique de l'hébergement).
 - Détails : Chaque animal possède une fiche d'identité avec son historique médical et ses mouvements.
- Actions Sensibles :
 - Soins : Saisie d'actes vétérinaires avec date de rappel.
 - Transfert : Déplacement d'un animal d'un refuge A vers un refuge B (gestion automatique de l'historique Héberge).
 - Adoption : Processus finalisant le départ de l'animal vers un particulier.
- Sécurité Renforcée : Pour toute modification critique (Ajout, Transfert, Adoption), une "Double Authentification" est requise : l'employé doit ressaisir ses identifiants dans une fenêtre modale pour valider l'action.

Tant pour la partie Administration et publique du site web, la navigation sur celui-ci est intuitive si l'utilisateur fait attention à tout ce qui est affiché à l'écran, ne menant donc à aucun cul-de-sac ou quoi que ce soit du genre...

2. SECTION TECHNIQUE : CONCEPTION DE LA BASE DE DONNÉES

Cette section reprend les choix de modélisation effectués lors des étapes précédentes (Modèle Conceptuel et Logique).

2.1. MODÈLE CONCEPTUEL DE DONNÉES (MCD)

Notre modèle s'articule autour de l'entité centrale **Animal**.

- Un **Animal** est identifié par un identifiant (qui dans la réalité pourrait être son tatouage ou une puce) et caractérisé par son espèce, race, âge, etc.
- Il est lié à un **Refuge** via une association datée (**Héberge**), permettant de tracer son historique complet (et pas seulement sa position actuelle), on stocke la date d'arrivée dans un refuge et de départ, si l'animal n'est jamais parti du refuge la date de départ vaut NULL, on sait donc qu'il est actuellement dans ce refuge (s'il n'est pas décédé)
- Les **Soins** sont gérés via une association ternaire entre un Animal, un Vétérinaire (Employé) et un Type de Soins, incluant une date.
- L'**Adoption** est traitée comme une entité à part entière liant un Animal et un Particulier, garantissant qu'un animal n'est adopté qu'une fois à un instant T, tout en gardant l'historique.

2.2. MODÈLE LOGIQUE DE DONNÉES (RELATIONNEL)

Le passage au relationnel a induit la création de tables spécifiques pour gérer les relations "Many-to-Many" et l'historique.

Points clés du schéma relationnel :

- **Table Héberge** : Clé primaire triple (`idAnimal`, `idRefuge`, `dateArrive`). La colonne `dateDepart` peut être `NULL` (signifiant que l'animal est actuellement présent). C'est une table essentielle pour la traçabilité.
- **Table Employe** : Contient un champ `login` et `motdepasse`. Les mots de passe sont stockés sous forme de `hash(pbkdf2:sha256)`, et non en clair, pour respecter les normes de sécurité.
- **Contraintes d'intégrité** : Utilisation de `CHECK` pour valider les dates (ex: `dateDepart > dateArrive`) et les statuts ('`Disponible`', '`Adopté`'). Beaucoup de contraintes `CHECK` sont faites pour vérifier les règles dites "métiers", des règles logiques du monde réel.

Ces descriptions du modèle conceptuel et du modèle logique de données ne sont pas exhaustives ni détaillées, car les deux étapes précédentes de ce projet y étaient spécifiquement consacrées.

3. DESCRIPTION TECHNIQUE DES FONCTIONNALITÉS WEB & SQL

Le site est développé en **Python (Flask)** avec une base de données **PostgreSQL**. L'interface utilise `HTML5`, `CSS3` et `JavaScript` (`Leaflet.js` pour la mini-map).

Voici les fonctionnalités les plus intéressantes techniquement :

3.1. GESTION TRANSACTIONNELLE DE L'ADOPTION

L'adoption est l'opération la plus complexe car elle touche 3 tables simultanément. Si une étape échoue, tout doit être annulé (`Rollback`).

Fonctionnement SQL :

1. Vérification/Création du Particulier : Si le numéro de téléphone existe déjà, on récupère l'ID existant, sinon on crée un nouveau `Particulier`.
2. Mise à jour de l'Animal : Passage du statut à '`Adopté`'.
3. Clôture de l'Hébergement : Mise à jour de la table `Héberge` pour définir la `dateDepart` à aujourd'hui (l'animal quitte physiquement le refuge).
4. Enregistrement de l'Adoption : Insertion dans la table `Adoption`.

Nous avons décidé d'implémenter la table `Adoption` afin d'envisager d'éventuelles améliorations du site web et de ses fonctionnalités. Par exemple, si un animal est ramené dans un refuge et adopté à nouveau, nous pourrions conserver son ancienne identité au lieu de créer un nouvel enregistrement. La table `adoption` nous permettrait de gérer ce genre de scénario. Bien que cette fonctionnalité ne soit pas actuellement nécessaire, elle représente une piste d'amélioration potentielle. En résumé, la présence ou non de la table `Adoption` n'a pas d'impact sur la base de données pour le moment.

Code Python/SQL associé :

```
try:
    # 1. Mise à jour statut
    cur.execute("UPDATE Animal SET statut = 'Adopté' WHERE id = %s", (id_animal,))
    # 2. Fermeture de l'hébergement actuel
    cur.execute("""
        UPDATE Heberge SET dateDepart = %s
        WHERE idAnimal = %s AND dateDepart IS NULL""",
        (date_jour, id_animal))
    # 3. Création lien adoption
    cur.execute("INSERT INTO Adoption ...")
    conn.commit() # Validation de la transaction
except:
    conn.rollback() # Annulation en cas d'erreur
```

3.2. PAGINATION ET TRI DYNAMIQUE

Pour gérer l'affichage de milliers d'animaux, nous avons implémenté une pagination côté serveur (SQL) plutôt que côté client, pour des raisons de performance (Si on devait sélectionner des milliers d'enregistrements, cela prendrait plus de temps, donc on sélectionne une petite portion correspondant à la page sur laquelle on est, puis on affiche)

La Requête SQL : Elle utilise **LIMIT** et **OFFSET** calculés dynamiquement en fonction du numéro de page. Le **ORDER BY** est également dynamique (sécurisé via une liste blanche de colonnes pour éviter les injections SQL).

Extrait d'une requête SQL :

```
SELECT A.*, H.dateArrive
FROM Animal A
JOIN Heberge H ON A.id = H.idAnimal
WHERE H.idRefuge = %s AND H.dateDepart IS NULL
ORDER BY A.nom ASC -- Dynamique (Nom, Espèce, Age...)
LIMIT 8 OFFSET 0; -- Dynamique (Page 1 = Offset 0, Page 2 = Offset 8...)
```

3.3. SÉCURITÉ : HACHAGE ET DOUBLE AUTHENTIFICATION

La sécurité est une priorité.

1. Stockage : Utilisation de la bibliothèque `werkzeug.security` pour hacher les mots de passe. À la création de la base de données, ce système n'était pas en place, nous avons donc conçu un programme pour hacher une première fois tous les mots de passe de la base de données. Dès lors, plus aucun mot de passe n'est accessible ni par les administrateurs du site web et de la BDD (nous) ni par l'utilisateur. Notre site ne gérant pas la création de nouveaux comptes "Employé" depuis le site directement, nous n'avons pas eu à gérer la sécurité quant à l'insertion de nouveaux comptes dans la base de données...
2. Double Auth : Sur les routes sensibles, un formulaire modal demande de ressaisir login/mdp. Une fonction Python `verifier_securite_hash(cur)` est appelée avant toute insertion SQL. (Ce formulaire apparaissant comme un pop-up a été implémenté grâce à un peu de code JavaScript intégré à la page HTML (Pas de script à part car pas assez de script pour que cela soit pertinent).

3.4. RECHERCHE AVANCÉE ET CARTE INTERACTIVE

La page d'accueil combine une recherche textuelle et une visualisation géographique.

- SQL : Utilisation de `LIKE %...%` sur plusieurs colonnes (nom, ville, code postal) avec `OR`, cette syntaxe nous permet donc de chercher largement, nous ne sommes pas obligés de rechercher le nom exact d'une ville ou le code postal exact d'un département, les premiers caractères permettent une recherche déjà relativement précise.
- JSON & Leaflet : Le backend Python génère une structure JSON contenant les coordonnées GPS (`lat`, `lon`) des refuges filtrés, qui est ensuite injectée dans le JavaScript pour placer les marqueurs sur la carte dynamiquement. Nous avons donc pour cette fonctionnalité dû légèrement modifier la base de données en ajoutant à la table `Refuge`, deux attributs, `lat` et `lon`, représentant respectivement les latitudes et longitudes géographiques des refuges (les localisations des refuges sont pour l'instant fictives, nous n'avons pas renseigné de vrais refuges). Leaflet est donc un projet Open Source qui nous a permis d'intégrer cette fonctionnalité à notre site web, il est assez simple d'utilisation (pas d'API nécessaires contrairement à Google Maps) et surtout est gratuit, là où Google Maps ne l'est pas.

3.5 ANALYSE D'UNE VUE ASSEZ COMPLEXE ET INTÉRESSANTE

Vue SQL :

```
CREATE VIEW V_Top5_Refuges_Transferts_Sortants AS
SELECT R.idRefuge, R.nom AS nomRefuge, COUNT(H.idAnimal) AS
nombreTransfertsSortants
FROM Refuge R
NATURAL JOIN Heberge H
WHERE H.dateDepart IS NOT NULL AND H.dateDepart >= (CURRENT_DATE -
INTERVAL '2 year')
GROUP BY R.idRefuge, R.nom
ORDER BY nombreTransfertsSortants DESC
LIMIT 5;
```

Fonctionnalité (Objectif) : Cette fonctionnalité permet d'afficher sur le tableau de bord le « Top 5 » des refuges les plus actifs, c'est-à-dire ceux ayant réalisé le plus grand nombre de transferts d'animaux vers d'autres structures au cours d'une période récente.

Intérêt Technique (Justification) :

1. **Agrégation (COUNT + GROUP BY) :** L'objectif n'est pas de restituer la liste exhaustive des transferts, mais plutôt de quantifier leur nombre par refuge. L'utilisation de la fonction `COUNT`, combinée à une clause `GROUP BY` sur le champ identifiant le refuge, permet d'obtenir cette agrégation.
2. **Filtrage Temporel (INTERVAL) :** L'intégration de la fonction `CURRENT_DATE - INTERVAL '2 year'` confère à la requête une nature dynamique. Ce filtrage temporel, assimilable à une fenêtre glissante, limite l'analyse aux transferts récents, excluant ainsi les données obsolètes relatives à des transferts effectués il y a plus de deux ans.

3. **Condition de sortie** : La condition technique "dateDepart IS NOT NULL" garantit que la requête ne prend en compte que les transferts effectivement réalisés, c'est-à-dire ceux pour lesquels l'hébergement est clos, et donc que la date de départ est renseignée.

4. PISTES D'AMÉLIORATIONS

Malgré la richesse fonctionnelle, plusieurs évolutions sont envisageables :

- **Upload de Photos** : Actuellement, le système gère des icônes ou des URLs. L'ajout d'un système d'upload de fichiers (stockage local ou S3) rendrait les fiches "animal" plus attractives.
- **Gestion des Stocks** : Ajouter une gestion des stocks de nourriture et de médicaments pour chaque refuge.
- **Système de Rôles Granulaire** : Distinguer les droits entre "Bénévole" (lecture seule), "Vétérinaire" (soins uniquement) et "Gérant" (accès total).
- **Alertes Automatiques** : Envoi d'emails automatiques aux vétérinaires lorsque la date de rappel d'un vaccin approche (actuellement visible uniquement sur le dashboard).
- **Prise de rendez-vous** : pouvoir depuis le site-web, pour un particulier, faire une demande de rendez-vous pour faire une première rencontre avec un animal par exemple. Puis ces demandes seront directement affichées sur le dashboard ou dans une section particulière de l'espace employé, pour pouvoir les traiter.
- **Pour une gestion plus efficace des adoptions et retours d'animaux** : comme évoqué précédemment dans le rapport, l'implémentation de la table `Adoption` dans la base de données actuelle serait une amélioration notable. Cela permettrait d'éviter les redondances d'informations et la création d'enregistrements inutiles dans les cas d'adoptions infructueuses.

5. ORGANISATION DU TRAVAIL

Nous avons donc réalisé ce projet en binôme avec une répartition des tâches la plus équilibrée possible, chacun a pu toucher à toutes les parties de la conception du site web.

Nous avons donc travaillé de manière à répartir le travail sur des tâches équivalentes, nous travaillions souvent ensemble pour être coordonnés, chacun développait des pages différentes mais dans leur entièreté (requêtes SQL, gestion routes (script python), page html, script javascript et feuille de style (css) associés) :

- Armand :
 - Page d'accueil avec affichage de la liste des refuges et mini map.
 - Page des pensionnaires dans le dashboard et fiche sur les animaux (accessible depuis la page des pensionnaires)
 - Page d'ajout d'animaux, de soins, d'adoptions, de transferts accessibles depuis la page du dashboard
 - Système de hachage des mots de passe, et actualisation de la fonction de login en conséquence

- Ramy :
 - Page de détail d'un refuge accessible en cliquant sur un refuge de la page d'accueil
 - Page de login avec système de vérification des sessions
 - Page principale du dashboard incluant tous les onglets présents sur la page et fonctionnalités associées.
 - Page de détail d'un refuge depuis le dashboard

- Travail commun :
 - Conception du MCD (Modèle Conceptuel de Données) et MLD (Modèle Logique de Données) de la base de données
 - Mise en place du cahier des charges (informel)
 - Décision de la logique et esthétique du site web

Bien évidemment, quand nous mentionnons "page" dans la répartition du travail fait, nous incluons toutes les fonctionnalités associées à la page, route dans le script python, requête SQL, page HTML, style CSS, script JavaScript...

Fin du rapport